

문자열의 모든 글자 뒤에 \$를 붙여서 출력하는 코드

```
ss = '파이썬짱!'

sslen = len(ss)
for i in range(0, sslen) :
    print(ss[i] + '$', end='')

파$I$션$짱$!$
```

프로그램 1: 문자열을 입력받아 반대로 출력

```
inStr, outStr = "", ""
count, i = 0, 0

inStr = input("문자열을 입력하세요: ")
count = len(inStr)

for i in range(0, count) :
    outStr += inStr[count - (i + 1)]

print(f"내용을 거꾸로 출력 --> {outStr}")

내용을 거꾸로 출력 --> !dlrow olleh
```

대문자와 소문자 변환하기

```
ss = 'Python is Easy. 그래서 programming 이 재미있습니다. ^^'

print(ss.upper())
print(ss.lower())
print(ss.swapcase())
print(ss.title())

PYTHON IS EASY. 그래서 PROGRAMMING 이 재미있습니다. ^^
python is easy. 그래서 programming 이 재미있습니다. ^^
pYTHON IS eASY. 그래서 PROGRAMMING 이 재미있습니다. ^^
Python Is Easy. 그래서 Programming 이 재미있습니다. ^^
```

문자열 찾기

문자열 메서드 한눈에 보기

메서드	기능	반환값	못 찾으면
count()	문자열 개수	개수(int)	0
find()	첫 위치 찾기	인덱스	-1
rfind()	마지막 위치 찾기	인덱스	-1

메서드	기능	반환값	못 찾으면
<code>index()</code>	첫 위치 찾기	인덱스	<code>ValueError</code>
<code>rindex()</code>	마지막 위치 찾기	인덱스	<code>ValueError</code>
<code>startswith()</code>	시작 여부 확인	<code>True / False</code>	<code>False</code>
<code>endswith()</code>	끝 여부 확인	<code>True / False</code>	<code>False</code>

`count()`

```
text = "banana"

print(text.count("a"))      # 3
print(text.count("na"))    # 2
print(text.count("x"))     # 0
```

`find()`

```
text = "banana"

print(text.find("a"))      # 1
print(text.find("na"))    # 2
print(text.find("x"))     # -1
```

`rfind()`

```
text = "banana"

print(text.rfind("a"))    # 5
print(text.rfind("na"))  # 4
print(text.rfind("x"))   # -1
```

`index()`

```
text = "banana"

print(text.index("a"))    # 1
print(text.index("na"))  # 2

# print(text.index("x"))
# ValueError
```

```

rindex()
text = "banana"

print(text.rindex("a"))    # 5
print(text.rindex("na"))  # 4

# print(text.rindex("x"))
# ValueError

```

```

startswith()
text = "Python Programming"

print(text.startswith("Python"))    # True
print(text.startswith("Java"))      # False

```

```

endswith()
text = "Python Programming"

print(text.endswith("Programming"))    # True
print(text.endswith("Python"))         # False

```

비교

비교	차이
find() vs index()	find() → -1, index() → ValueError
rfind() vs rindex()	rfind() → -1, rindex() → ValueError
startswith()	시작 여부 확인
endswith()	끝 여부 확인

핵심 암기

메서드	기억할 것
count()	개수 반환
find()	첫 위치, 없으면 -1
rfind()	마지막 위치, 없으면 -1
index()	첫 위치, 없으면 오류
rindex()	마지막 위치, 없으면 오류
startswith()	시작 여부
endswith()	끝 여부

암기 팁

- `find` → -1
- `index` → `ValueError`
- `r` → 오른쪽(뒤쪽)부터 검색
- `start` → 시작 확인
- `end` → 끝 확인

```
ss = '파이썬 공부는 즐겁습니다. 물론 모든 공부가 다 재미있지는 않죠. ^^'

print(ss.count('공부'))
print(ss.find('공부'), ss.rfind('공부'), ss.find('공부', 5), ss.find('없다'))
print(ss.index('공부'), ss.rindex('공부'), ss.index('공부', 5))
print(ss.startswith('파이썬'), ss.startswith('파이썬', 10), ss.endswith('^^'))

2
4 21 21 -1
4 21 21
True False True
```

문자열이 괄호로 감싸 있지 않으면 괄호로 감싸주는 프로그램

```
ss = input("입력 문자열 ==> ")
print("출력 문자열 ==>", end='')

if ss.startswith('(') == False :
    print("(", end='')

print(ss, end='')

if ss.endswith(')') == False :
    print(")", end='')

출력 문자열 ==> (파이썬 열종 중~~)
```

문자열 공백 삭제/변경하기

문자열 메서드 (`strip`, `rstrip`, `lstrip`, `replace`)

메서드	기능	반환값
<code>strip()</code>	양쪽 공백(또는 지정 문자) 제거	새로운 문자열
<code>lstrip()</code>	왼쪽 공백(또는 지정 문자) 제거	새로운 문자열
<code>rstrip()</code>	오른쪽 공백(또는 지정 문자) 제거	새로운 문자열
<code>replace()</code>	문자열 변경	새로운 문자열

`strip()`

양쪽의 공백을 제거합니다.

```
text = "  Python  "
print(text.strip())
```

결과

Python

지정한 문자도 제거할 수 있습니다.

```
text = "###Python###"
print(text.strip("#"))
```

결과

Python

`rstrip()`

왼쪽 공백만 제거합니다.

```
text = "  Python  "
print(text.rstrip())
```

결과

Python

지정한 문자도 제거할 수 있습니다.

```
text = "###Python###"
print(text.rstrip("#"))
```

결과

Python###

`rstrip()`

오른쪽 공백만 제거합니다.

```
text = "  Python  "
print(text.rstrip())
```

결과

Python

지정한 문자도 제거할 수 있습니다.

```
text = "###Python###"
print(text.rstrip("#"))
```

결과

###Python

`replace()`

특정 문자열을 다른 문자열로 변경합니다.

```
text = "I like Python"
print(text.replace("Python", "Java"))
```

결과

I like Java

모든 문자열을 변경합니다.

```
text = "banana"
print(text.replace("a", "*"))
```

결과

b*n*n*

변경 횟수를 지정할 수도 있습니다.

```
text = "banana"
print(text.replace("a", "*", 2))
```

결과

```
b*n*na
```

비교표

메서드	제거 위치
<code>strip()</code>	양쪽
<code>rstrip()</code>	왼쪽
<code>lstrip()</code>	오른쪽
<code>replace()</code>	문자열 변경

핵심 암기

메서드	기억할 것
<code>strip()</code>	양쪽 제거
<code>rstrip()</code>	왼쪽 제거
<code>lstrip()</code>	오른쪽 제거
<code>replace()</code>	문자열 변경

암기 팁

- **rstrip** → Right(오른쪽)
- **lstrip** → Left(왼쪽)
- **strip** → 양쪽 제거
- **replace** → 문자열 바꾸기

```
ss = '파이썬'
print(ss.strip())
print(ss.rstrip())
print(ss.lstrip())
```

```
ss = '열심히 파이썬 공부 중~~'
print(ss.replace('파이썬', 'Python'))
```

```
파이썬
['파', '이', '썬']
파이썬
열심히 Python 공부 중~~
```

```
ss = '---파---이---썬---'
print(ss.strip('-'))
```

```
ss = '<<<파<<0>>>썩>>>'
print(ss.strip('<>'))
```

```
파---0---썩
파<<0>>>썩
```

문자열中间的 공백까지 삭제해주는 코드

```
inStr = "    한글 Python 프로그래밍  "
outStr = ""

for i in range(0, len(inStr)) :
    if inStr[i] != ' ':
        outStr += inStr[i]

print("원래 문자열 ==> " + '[' + inStr + ']')
print("공백 삭제 문자열 ==> " + '[' + outStr + ']')
```

```
원래 문자열 ==> [    한글 Python 프로그래밍  ]
공백 삭제 문자열 ==> [한글 Python 프로그래밍]
```

문자열을 입력받아 그 중 o를 \$로 변경하는 문자열 변경을 응용

```
ss = input("입력 문자열 ==> ")

print("출력 문자열 ==>", end='')
for i in range(0, len(ss)) :
    if ss[i] != 'o' :
        print(ss[i], end='')
    else :
        print('$', end='')

print(f"\n한 줄로 처리한 결과: {ss.replace('o', '$')}")
```

```
출력 문자열 ==>IT C$$kB$$k f$r Pyth$n
한 줄로 처리한 결과: IT C$$kB$$k f$r Pyth$n
```

문자열 분리 결합치기

문자열 메서드 한눈에 보기

메서드	기능	반환값
<code>split()</code>	문자열 분리	리스트(list)
<code>splitlines()</code>	줄바꿈 기준 분리	리스트(list)
<code>join()</code>	문자열 합치기	새로운 문자열(str)

`split()`

구분자를 기준으로 문자열을 분리합니다.

```
text = "apple,banana,grape"
print(text.split(","))
```

결과

```
['apple', 'banana', 'grape']
```

공백 기준으로 분리

```
text = "Python is easy"
print(text.split())
```

결과

```
['Python', 'is', 'easy']
```

`splitlines()`

줄바꿈(\n)을 기준으로 문자열을 분리합니다.

```
text = "Python\nJava\nC"
print(text.splitlines())
```

결과

```
['Python', 'Java', 'C']
```

`join()`

리스트의 문자열을 하나의 문자열로 합칩니다.

```
fruit = ["apple", "banana", "grape"]
print(",".join(fruit))
```

결과

```
apple,banana,grape
```

공백으로 합치기

```
words = ["Python", "is", "easy"]  
print(" ".join(words))
```

결과

```
Python is easy
```

비교

메서드

기능

<code>split()</code>	문자열 → 리스트
<code>splitlines()</code>	줄바꿈 기준 문자열 → 리스트
<code>join()</code>	리스트 → 문자열

핵심 암기

메서드

기억할 것

<code>split()</code>	문자열 분리
<code>splitlines()</code>	줄바꿈 기준 분리
<code>join()</code>	리스트를 문자열로 합치기

암기 팁

- `split` → 문자열 → 리스트
- `join` → 리스트 → 문자열
- `splitlines` → 줄바꿈(\n) 기준 분리

```
ss = 'Python을 열심히 공부 중'  
print(ss.split())
```

```
ss = '하나:둘:셋'  
print(ss.split(':'))
```

```
ss = '하나\n둘\n셋'  
print(ss.splitlines())
```

```
ss = '%'  
print(ss.join('파이썬'))
```

```
['Python을', '열심히', '공부', '중']  
['하나', '둘', '셋']  
['하나', '둘', '셋']  
파%이%썬
```

연/월/일 형식으로 문자열을 입력받아 10 년 후 날짜를 출력하는 코드

```
ss = input("날짜(연/월/일) 입력 ==> ")
ssList = ss.split('/')

print("입력한 날짜의 10 년 후 ==> ", end='')
print(f"{int(ssList[0]) + 10}년", end='')
print(ssList[1] + "월", end='')
print(ssList[2] + "일")

입력한 날짜의 10 년 후 ==> 2029 년 12 월 31 일

before = ['2019', '12', '31']
after = list(map(int, before))
print(after)

[2019, 12, 31]
```

문자열 정렬/채우기

문자열 메서드 한눈에 보기

메서드	기능	반환값
<code>center()</code>	가운데 정렬	새로운 문자열
<code>ljust()</code>	왼쪽 정렬	새로운 문자열
<code>rjust()</code>	오른쪽 정렬	새로운 문자열
<code>zfill()</code>	앞을 0 으로 채움	새로운 문자열

`center()`

지정한 길이만큼 가운데 정렬합니다.

```
text = "Python"

print(text.center(10))
print(text.center(10, "*"))
```

결과

```
Python
**Python**
```

`ljust()`

지정한 길이만큼 **왼쪽 정렬**합니다.

```
text = "Python"
print(text.ljust(10))
print(text.ljust(10, "-"))
```

결과

```
Python
Python----
```

`rjust()`

지정한 길이만큼 **오른쪽 정렬**합니다.

```
text = "Python"
print(text.rjust(10))
print(text.rjust(10, "-"))
```

결과

```
Python
----Python
```

`zfill()`

앞쪽을 **0**으로 채웁니다.

```
print("25".zfill(5))
print("123".zfill(6))
```

결과

```
00025
000123
```

비교

메서드

`center()`

기능

가운데 정렬

메서드	기능
<code>ljust()</code>	왼쪽 정렬
<code>rjust()</code>	오른쪽 정렬
<code>zfill()</code>	앞을 0 으로 채움

핵심 암기

메서드	기억할 것
<code>center()</code>	가운데 정렬
<code>ljust()</code>	왼쪽 정렬
<code>rjust()</code>	오른쪽 정렬
<code>zfill()</code>	앞을 0 으로 채움

암기 팁

- `center` → 가운데
- `ljust` → Left(왼쪽)
- `rjust` → Right(오른쪽)
- `zfill` → Zero(0) 채우기

```
ss = '파이썬'

print(ss.center(10))
print(ss.center(10, '-'))
print(ss.ljust(10))
print(ss.rjust(10))
print(ss.zfill(10))
```

```
파이썬
---파이썬----
파이썬
      파이썬
0000000 파이썬
```